

Signals and Sources — an architecture note

Date: 2026-05-20 **Status:** Thinking out loud, not a plan **Reader:** print this, mull, come back with direction

The question

We have six things in the engine that produce signals:

- the **Tidal port** (pattern-of-events, mini-notation, Pattern algebra)
- the **fugue machine** (a Voice transform that lifts a subject into a Pattern)
- **Grids** (Olivier Gillet's drum brain as a BEAM `gen_server`)
- **René** (the Cartesian sequencer as a BEAM `gen_server`)
- **ZR / Repetitor** (rhythm-library player as a BEAM `gen_server`)
- **Polysignals** (typed declarations of autonomous FH-2 bank configs)

They feel different — different shapes, different syntax, different runtime. The question: is that difference *essential*, or is it an *artifact of how we built them*? And if it's an artifact, what's the underlying thing all six are special cases of?

This note argues: there is one underlying primitive. The differences between the six are nearly all evolutionary, not essential. If we name the primitive, the cross-machine weaving you described — René clocked by a pair of euclidean rhythms, Grids fills triggered by Tidal patterns, René notes quantisation controlled by PureScript, and "thousands of other combination potentials" — falls out as ordinary composition rather than special-cased plumbing.

Three categories, two of which dissolve

Earlier I proposed three categories:

1. **Pure pattern** (Tidal, fugue) — **Arc** → **Array Event**
2. **Virtual module** (Grids, René, ZR) — stateful `gen_server`, ticks on a clock
3. **Hardware-delegated** (polysignals) — declaration → FH-2 firmware does the work

You pushed back on both 1-vs-2 and on whether category 3 even needs to exist as a separate category. I think you're right on both counts.

Pure pattern vs virtual module — accidental, not essential

Grids, René, and ZR are `gen_servers` today because of two historical reasons, neither of them deep:

- **They consume Link beats.** Their "tick" is external to whatever Pattern algebra would describe them. So we built them as processes that subscribe to a clock and emit on it.
- **They have parameters that mutate from MIDI controllers.** We built the live-control bus *after* the first vmods, so the natural shape at the time was "gen_server with internal state mutated by external messages."

But the live-control bus is now the thing that holds mutable parameters, and the Pattern algebra is already perfectly capable of consuming a clock — the walker queries Patterns based on Link time. So both reasons have evaporated.

Concretely:

- **Grids** is a deterministic function of (`x`, `y`, `fillBd`, `fillSd`, `fillHh`, `randomness`, `step_in_pattern`). Internal "randomness" is comparison against a 32-step deterministic noise table — already a pure function of step position. **Grids can be a Pattern DrumHit** that reads its parameters from the live-control bus.
- **ZR** is even more obviously a Pattern: it's a static array indexed by step. Different libraries are different arrays. There's no state to speak of.
- **René** is harder because of the wandering cursor with skip/glide, but the cursor position at step `N` is a pure function of (`navMode`, `skip[0..N]`, `glide[0..N]`, `stepsPerCycle`, `N`). No actual path-dependence, just a stateful-looking computation. **Also a Pattern.**

So categories 1 and 2 collapse: **every event-emitting thing is a Pattern**, and the `gen_server` implementation is an *optimisation strategy* (no recomputation per query, isolated random state, faster restart), not a different kind of thing.

This matters because Pattern algebra is closed: `<>`, `cat`, `fastCat`, `every`, `jux`, `arrange`, the lot. If Grids is a Pattern, then `every 4 (rev) studioGrids` works. If René is a Pattern, then `cat [studioRene, studioGrids]` is a valid composition. None of that is true today — the vmods sit outside the Pattern algebra.

Polysignals — split spec from realisation

Your second push: polysignals are tied to FH-2 today, but they don't have to be. The polysignal declaration (8 LFOs at these ratios) is independent of where the actual oscillation happens. Possibilities:

- **FH-2 firmware** (current) — send SysEx, FH-2 generates
- **ES-9 + Rust** (planned in `[[reference_modulation_module_research]]`) — we run the LFO math ourselves and stream samples to ES-9 jacks
- **Virtual output** (not yet imagined) — the LFO runs in BEAM and writes its value to a live-control bus key

So polysignal has the same spec-vs-realisation split as Patterns have: the *declaration* says "what should be produced", the *runtime* picks how. A polysignal whose realisation is "FH-2 firmware" is just one strategy among several. The "hardware-delegated" category isn't a separate kind — it's a realisation choice.

Polysignals as virtual modules

This is your generative twist, and I think it's the productive one.

what if polysignals existed that were only virtual outputs that plugged in to virtual modules?

Concretely: suppose `polyLfo (Virtual "lfoBank") [...]` is a polysignal realisation whose 8 outputs aren't FH-2 jacks but **live-control bus keys**: `lfoBank.0` through `lfoBank.7`. The LFO math

runs in a BEAM gen_server. Every 10ms or 20ms it writes its current value to each key. Nothing physical happens.

Now wire it in:

```
studioGrids = grids iac 12 $ gridsConfig
{ x      = liveIntOr 128 "lfoBank.0"  -- LFO 1 modulates Grids X
, y      = liveIntOr 128 "lfoBank.1"  -- LFO 2 modulates Grids Y
, fillHh = liveIntOr 200 "lfoBank.2"  -- LFO 3 modulates HH density
, ...
}
```

You have generative drum-machine modulation, **end-to-end in software**, with no FH-2 in the loop. The polysignal becomes the slow-parameter generator that plugs into the fast event generators.

And the original physical-output polysignal is the same thing with a different realisation:

```
polyLfo (BankMain) [...]      -- realised by FH-2 firmware
polyLfo (BankCv 2) [...]      -- realised by FH-2 on expander 2
polyLfo (Es9 [0,1,2,3]) [...] -- realised by our cv-router
polyLfo (Virtual "lfoBank") [...] -- realised by BEAM, writes to bus
```

The declaration is identical. The realisation chooses where the energy goes.

This is, I think, the unification that makes the whole architecture fall into place.

The single primitive: Source

If categories 1 and 2 dissolve, and category 3 splits into spec + realisation, then what's left? **One primitive: a thing that produces values over time, with a name on it.**

Call it a **Source**. A Source has:

- a **type** — `Int`, `Bool`, `Note`, `DrumHit`, `Array Int`, `Unit` (clock pulse), `Voltage`, ...
- a **time domain** — discrete events (Pattern of triggers), continuous signal (an LFO), or one-shot (a polysignal install)
- a **realisation strategy** — pure Pattern queried by the walker, BEAM gen_server, hardware delegation (FH-2 firmware, cv-router), virtual bus writer

The complement is a **Sink** — anything that consumes Sources:

- **Live-control bus keys** (`grids.x`, `lfoBank.0`, ...) — the shared parameter plane
- **Voice destinations** (IAC ch 1, cv-router voice "bass1", ...) — the audible boundary
- **Polysignal banks** (FH-2 main, FH-2 cv2, ...) — the hardware-config boundary
- **Vmod parameter slots** (René's `notes` array, Grids' `randomness`, ...) — currently a wrapper around live-control reads, but worth naming

Wiring is **Source → Sink**. Composition is wiring graphs. The whole engine becomes a system that *resolves a wiring graph* — picks realisations for each Source, claims Sinks, runs the resulting BEAM processes.

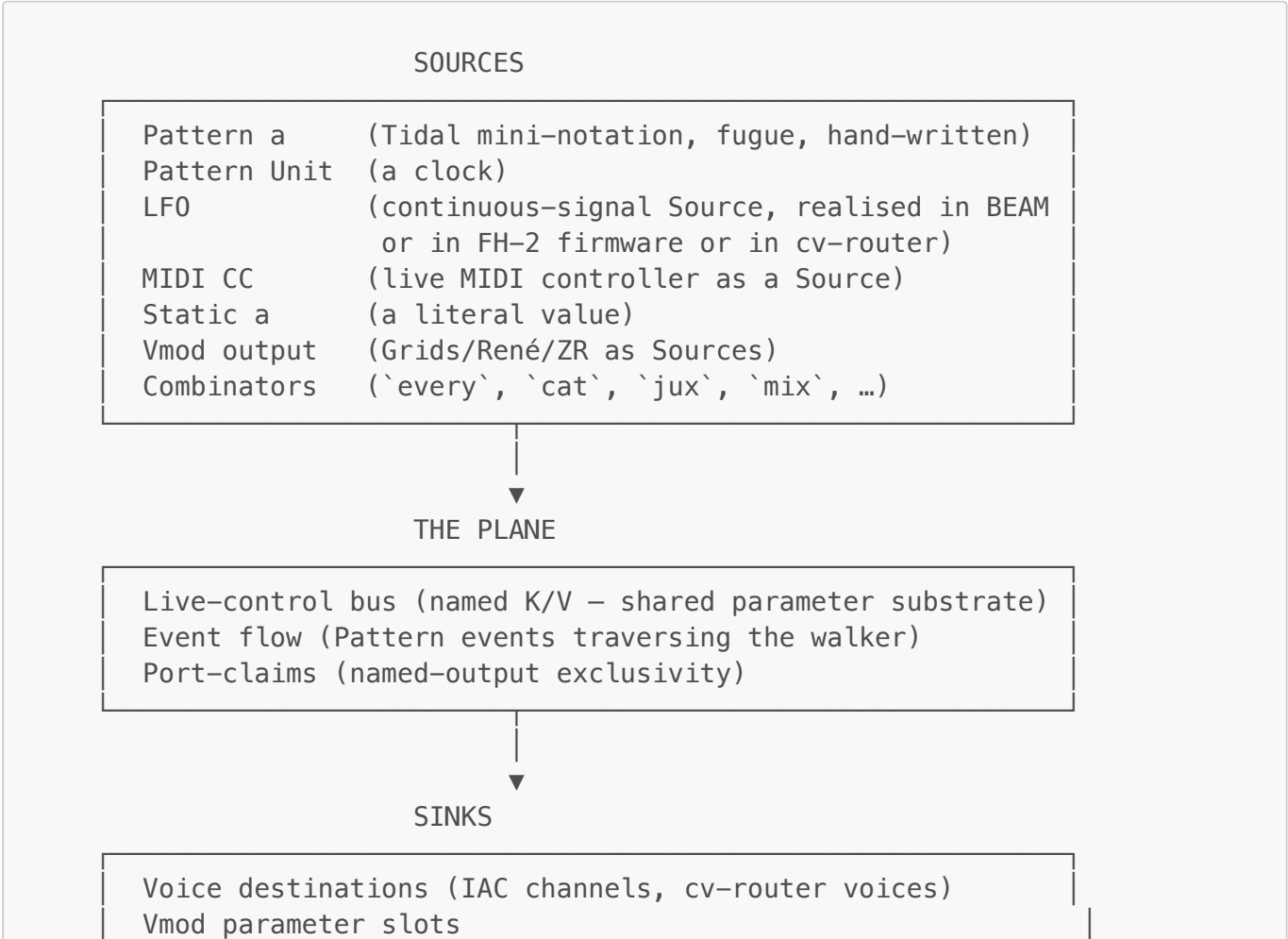
Why this matters

The three weaving examples you posed are different in the *current* architecture and trivially uniform in the proposed one:

Weaving	Today	In the unified picture
Grids fills triggered by a Tidal pattern	Doesn't exist; would need new combinator	A Pattern Source writes to grids.fillHh . Same as MIDI Twister already does.
René notes controlled by PureScript	Doesn't exist; would need new array-aware combinator	A Pattern Source writes to rene.note . Same shape as above.
René clocked by a pair of euclidean rhythms	Doesn't exist; needs vmod clock-source refactor	René consumes a Pattern Unit Source as its clock. Same shape as the others.

All three are the same thing: **a Source feeding a Sink**. The novelty in the unified picture is that *all parameter slots and the clock slot are Sinks*, accepting any Source the user wants to plug in.

The unified picture



```
Polysignal banks (FH-2 main / cvN / gtN; ES-9 jacks)
Other Sources (recursion: one machine's output is
                another's parameter)
```

The six emitters are now just different Source shapes plus different realisation choices. The notations stay distinct on the surface (Tidal mini-notation, polysignal declarations, vmod configs) because each is ergonomically suited to its niche — but they all *desugar* to the same Source/Sink algebra underneath.

What live coding looks like in this frame

The user writes:

- **Tidal patterns** — fast event Sources for melodic/rhythmic content
- **Polysignal declarations** — slow parameter Sources, hardware or virtual
- **Vmod configs** — pre-rolled event Sources (Grids/René/ZR) for genre-shaped rhythm and pitch
- **Wirings** — `controlFromPattern "grids.fillHh" (cat [pure 80, pure 220]), or
clockedBy (euclid 3 8) studioRene`

The MIDI controllers are *a/so* Sources, plugged into the same Sinks. Two Sources contesting the same Sink is a claim conflict, surfaced at install time, resolved by the user (override, mix, blend, fall-through).

The compositional language gets one new primitive — Source → Sink wiring — and everything else stays as it is. The Tidal Pattern algebra extends naturally because Patterns are Sources, and Sinks just become new consumers.

And the "thousands of other combination potentials" become *visible* as wiring choices rather than hidden in per-emitter implementation quirks. The IDE-for-music framing ([[project_purerl_tidal_ide_framing]]) sharpens: the IDE is showing you the wiring graph, letting you edit it, catching conflicts at install time.

The smallest next step

The cleanest demonstration of the unified picture, with the smallest code surface, is **the virtual polysignal**:

1. Add a `Virtual String` constructor to the polysignal Bank ADT
2. The fh2-daemon (or a new BEAM gen_server — probably the latter) ticks a virtual LFO at, say, 50 Hz and writes its current value to a live-control bus key named after the polysignal's slug + output index
3. Use it: `polyLfo (Virtual "lfoBank") [...]`, then wire a Grids parameter to `lfoBank.0` via the existing `liveIntOr`

This requires:

- One new ADT case
- One new BEAM gen_server (the virtual realiser)
- Zero changes to Grids, René, ZR, Tidal, the walker, Calypso, anything user-facing

It's a contained PR that **demonstrates the spec/realisation split is real**: the same polysignal declaration syntax produces a hardware signal or a virtual signal depending on the Bank. Once that lands, the generative-music end of the picture (LFOs modulating drum machines) is proven out, and we have something to live-code with while we sort out the bigger Pattern-ification of the vmods.

The bigger refactor — making Grids/René/ZR proper Patterns at the algebra boundary, even while keeping the `gen_server` runtime — can happen incrementally after.

What I'm uncertain about

Three things that need your thinking, not mine:

1. **Is the Source/Sink vocabulary right?** It risks importing Eurorack patch-cable mental models that don't quite fit BEAM message flow. But I haven't found cleaner words.
2. **How aggressive to be about Pattern-ifying the existing vmods.** The current `gen_server` implementation is fast and works. Forcing it through a Pattern interface adds a layer. Maybe the right shape is: *the algebra* treats them as Patterns; *the runtime* stays as `gen_servers` and exposes a Pattern-shaped query interface. That feels like the cheapest unification.
3. **Whether the live-control bus is the right plane.** Right now it's ETS-backed K/V, optimised for cheap reads. If polysignals start writing to it at 50 Hz across 64+ keys, plus MIDI controllers, plus Patterns-as-Sources, we may bump into the bus' design assumptions. The architecture might want a richer plane — typed channels, maybe even something pub/sub-shaped — though I'd start by stressing the bus and only then redesigning.

Print, mull, return

The two productive critiques you raised this morning re-shaped the note's central claim. Everything else here is downstream of those two insights. Take it, mark it up, push back where it's wrong, and we'll decide a direction from your annotations.

Annotations from first read (2026-05-20, afternoon)

One-shot is `const x`

The polysignal-install case I described as "one-shot" isn't a third time-domain — it's a Source whose value happens to be constant after emission. `const x`, in the same algebra. Which means the three-way split I had earlier (events / continuous / one-shot) collapses to one: every Source is a function `time → value`; the shape of the function (delta-pulses, smooth signal, constant) is just *what kind of function*, not what kind of thing.

This is a stronger unification than I claimed. It also tidies up hardware delegation: an FH-2 SysEx install is a constant-valued Source (the config) wired into a one-write Sink (the FH-2 config bank). The FH-2 firmware then *acts as a Source* itself, in the voltage domain, invisible to BEAM but the same primitive.

So: not three time-domains. One algebra. Different value-functions.

Source/Sink terminology — endorsed

Not Eurorack import; fundamental to computation. (Stream processing, dataflow languages, reactive programming, FRP, even shell pipes.) Keep it.

Pattern-ification — a compile-step architecture

This is now the open question I think is most interesting to develop. The shape Andrew points at: an explicit compile step that goes from the Source/Sink algebra (PureScript) to *some* runtime expression. The current arrangement already has a compile step (PureScript → Erlang via purs-backend-erl + walker registration events → gen_server starts), but it's implicit and entangled with the language target.

Making it explicit opens a real design space:

- **Production runtime** — current: BEAM with per-voice gen_servers, ETS bus, walker, fh2-daemon. Optimised for low-latency rig use.
- **Single-node runtime** — everything in one Node.js event loop, no BEAM. Slower but simpler for learning, demos, or running without the rig.
- **Test runtime** — pure functions, capture events as data structures for assertion. The unit-test backend.
- **Headless render runtime** — compile a Section to MIDI file or audio stems. No real-time clock.
- **Visualising runtime** — slow, step-through, animated. Educational surface.

The Source/Sink algebra becomes the IR. The runtime is the target. The Pyrrhic-victory worry — "perfect abstraction, unusable live- coding" — gets defused by recognising that the *production* runtime doesn't have to be the *only* runtime, and the abstraction's quality is judged against all of them.

This is a much bigger ambition than "build a virtual polysignal," and it's the right framing to mull on.

Rust runtime as a divergence option

Nothing in the IR sketch requires PureScript — the constructors are closed sums, the records are flat, no laziness, no typeclasses inside the IR data itself. In principle the same IR could be consumed by a Rust runtime running at audio rate.

Parked, not pursued. PureScript/purerl is expected to be fine for control-rate work, which covers everything we currently need (LFOs at 50-100 Hz, vmod ticks at Link rate, MIDI events). Audio- rate sample generation on the cv-router is the only hypothetical use case, and we'd only look at it if bandwidth limits actually bite. Worth noting that the divergence-of-runtimes design is *available* without forcing us toward it.

Bus bandwidth — static analysis at compile time

Each Source has a known rate (Pattern: per-cycle samples, LFO: 50 Hz, MIDI CC: bounded). Each Sink has a known capacity. The bus has a known throughput. So $\sum \text{source.rate} \leq \text{bus.throughput}$ is computable at *compile* time — same compile step as above — and unreasonable configurations can warn or block before they ship.

This is the **port-claims framework extending from exclusivity to capacity**. Same shape (named-resource accounting at install time), additional dimension (not just "who owns it?" but "is the budget

exceeded?"). And it lives at the same layer in the architecture.

Both port-claims and bandwidth-analysis are static checks against the *same* IR. Which suggests the IR is real and worth designing.

The test — does this enable simplicity at the user level?

Architectural elegance is necessary but not sufficient. The ultimate test for everything in this exploration:

Does the abstraction make the user-facing music language simpler or more complex? If the IR types (Source, Sink, Wiring, ValueType) leak into what someone types in a Calypso cell, we've propagated complexity even when the internal code got cleaner.

The user-facing surface is cell-text — what someone types in Calypso to make sounds. Mini-notation, polysignal declarations, vmod configs, section combinators. That's the language whose shape we're protecting.

Applying the test to the three weaving examples this note proposed:

Weaving	User writes
Grids fills from a Tidal pattern	<code>gridsConfig { fillHh = sourcedBy (every 4 (const 220) (const 80)) }</code>
René notes from PureScript	<code>renewith { ..., notes = sourcedBy myNotePattern }</code>
René clocked by euclidean rhythm	<code>renewith { ..., clock = clockedBy (euclid 3 8) }</code>

Two new combinators (`sourcedBy`, `clockedBy`) unlock all three cross-machine weavings. The IR's Sources, Sinks, Wirings, and ValueTypes are invisible — they happen under the hood when these combinators desugar. The user surface stays compact.

Failure mode: if the same user had to write something like

```
wire (asPattern myFillPattern)
  (sinkRef "studioGrids" "fillHh")
  [ScaleLinear 0.0 1.0 0.0 255.0]
```

...then the IR has leaked. Even if the internal types are pristine, we've made the music language more complex.

The architecture is good only if all the unifications it enables fit through this kind of two-combinator surface. Track this test against every future proposal in this thread.

Where this leaves the architecture

Three things now landed:

1. **One algebra, not three categories.** Source/Sink with value-functions varying in shape. `const x` is one of those shapes.
2. **Compile step is the architecture's spine.** Algebra $\rightarrow$ IR $\rightarrow$ runtime, with multiple runtime targets possible.
3. **Static analysis (port-claims + bandwidth) lives at the IR.** Real safety net, computed before anything runs.

Still to decide:

- Whether the next concrete piece is still "virtual polysignal" (the cheap proof of spec/realisation split) or something further up the stack (a sketch of the IR shape, even if not implemented).
- How heavyweight to make the compile step. The cheap version is "just an annotated walker"; the rich version is a real IR with passes, sugarings, target-specific lowerings.
- Whether the simpler runtimes (single-node, headless) are a near-term goal or a long-tail one.

Print this revision, mull, return.